



UNIVERSITÀ DI PARMA

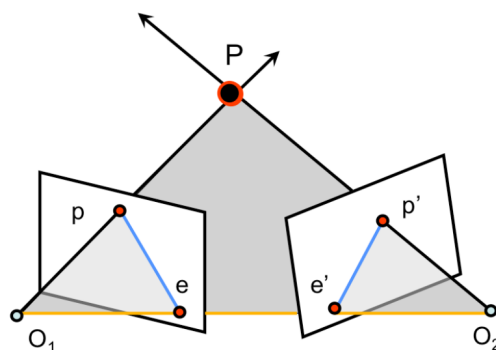
Stereo Matching



MATCH STEREO OVVERO RICERCA
CORRISPONDENZE

- Small recap about:
 - Epipolar geometry
 - Rectified Stereo Images
- Disparity
- Correspondences search
 - NCC
 - SAD

- [FP] D. A. Forsyth and J. Ponce. Computer Vision: A Modern Approach (2nd Edition). Prentice Hall, 2011.
- [HZ] R. Hartley and A. Zisserman. Multiple View Geometry in Computer Vision. Cambridge University Press, 2003.
- CS231A · Computer Vision: from 3D reconstruction to recognition
 - Prof. Silvio Savarese – Stanford University
- 15-463, 15-663, 15-862, Computational Photography, Fall 2021
 - Prof. Ioannis Gkioulekas – Carnegie Mellon Graphics Lab



- O_1O_2P : epipolar plane
- O_1O_2 : baseline
- pe & $p'e'$: epipolar lines (all epipolar lines meet in e or e')
- e & e' : epipoles
 - Intersection of baseline with image planes
 - Projection of O_1 & O_2

Un po' di definizioni

La linea che connette i due pin hole ovvero i due centri ottici è la linea epipolare

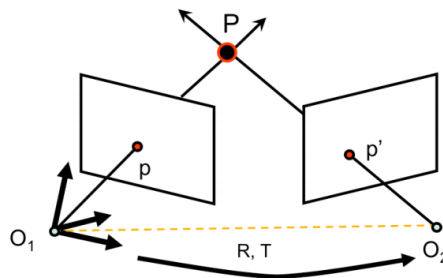
Chiamiamo il piano determinato dai centri ottici e da P come piano epipolare.

Questo piano interseca i piani immagine in due rette che sono le rette epipolari

e ed e' sono i punti di intersezione tra la baseline e i piani immagine

Al variare di P cambieranno:

- il piano epipolare
- le proiezioni p e p'
- le rette epipolari

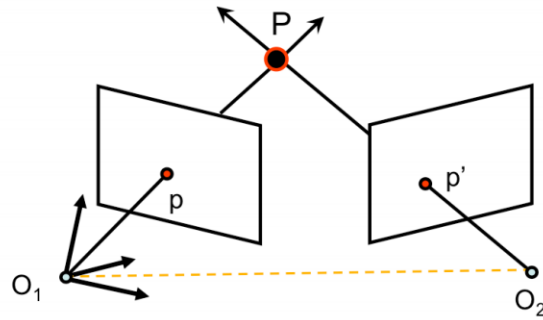


$$p^T \cdot [T \times (Rp')] = 0$$

$$p^T \cdot \underbrace{[T_x] R}_E p' = 0$$

- E is the **Essential Matrix**

La matrice essenziale lega i punti p e p' nel caso delle telecamere canoniche
Quello che conta ai fini di questa lezione è che $E p'$ mi rappresenta la linea
epipolare su cui cercare p se conosco p'



[Eq. 13]

$$p^T F p' = 0$$

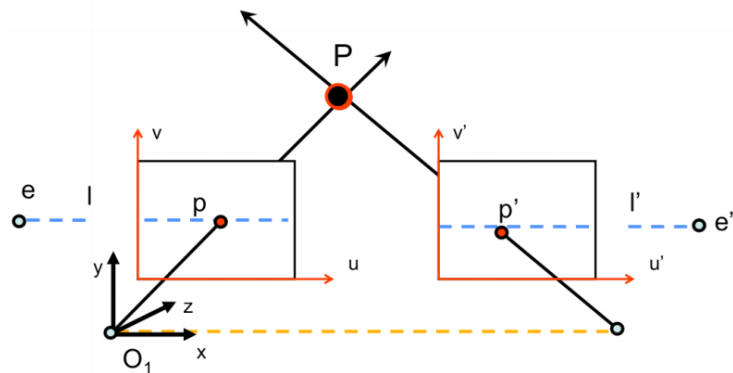
$$F = K^{-T} \cdot [T_x] \cdot R K'^{-1}$$

[Eq. 14]

- F is the **Fundamental Matrix** (Faugeras and Luong 1992)

Nel caso di telecamere non canoniche ricavo una relazione simile ovvero F

Special case: parallel image planes

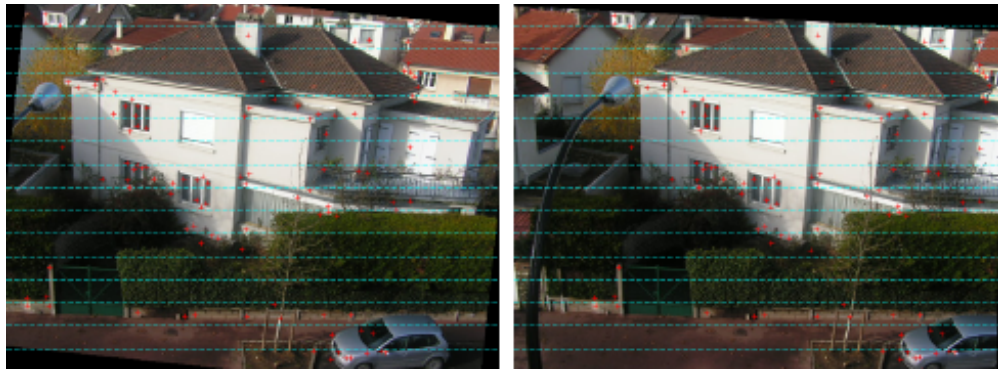


- $v=v'$ means the same line!
- The search for p' is simpler...

Se “rettifico” le immagini stereo p e p' giacciono sulla stessa linea e di conseguenza la ricerca di corrispondenze è ben più semplice.

Ma lo riesco a fare solo se conosco F (o E , K e K')

Special case: parallel image planes



- Example of two images whose planes are parallel to each other
- Epipolar lines are parallel to each other

Esempio di immagini rettificate

- If we know p and p' , their relative position and their lines of sight we can use them to **triangulate** and recover the unknown: **P**
- Steps:
 - **Camera geometry:** given a set of correspondences find camera parameters
 - **Correspondences:** for all points p in one image find correspondent p' in the other image
 - **Scene Geometry:** given a set of correspondences and parameters of both cameras reconstruct the 3D scene

Se conosco I due punti p e p' (o meglio tutti i p' corrispondenti a tutti I p della prima immagine) e la loro posizione relativa, io posso trinagolare. È un po' come se mi mettessi con il teodolite su P e calcolassi gli angoli con p e p' di cui conosco la posizione tra di loro. Va da sé che è facile capire la distanza di P , date le linee di vista con cui inquadro P lo posso anche fissare in una posizione ben precisa → ricostruzione 3D

Quindi i problemi aperti sono:

1. Devo conoscere la posizione relativa tra p e p' . Questo dipende dai parametri del sistema stereo. In questo pacco di slide vediamo cosa lega p e p' e accenniamo anche a come si trovano (c'è pacco di slide separato)
2. Per una ricostruzione 3D prendo tutti I punti p della prima immagine e li cerco nella seconda
3. dati i parametri delle telecamere e diverse corrispondenze p/p' Mi calcolo il 3D

In questo pacco di slide ci occupiamo degli ultimi 2 punti

- What is the difference between those images?



Esempi di immagini stereo



Quello che si nota è che gli stessi oggetti si trovano in posizioni leggermente differenti tra destra e sinistra

E la differenza di posizione DISPARITÀ è tanto maggiore tanto l'oggetto è vicino

Fate un test, collimate, tenendo chiuso un occhio il pollice a qualcosa di molto lontano. E poi chiudete/aprite alternativamente gli occhi. Potete notare come il pollice si sposti significativamente nelle due viste (è relativamente vicino a voi), mentre il punto lontano no...



Quello che si nota è che gli stessi oggetti si trovano in posizioni leggermente differenti tra destra e sinistra

E la differenza di posizione DISPARITÀ è tanto maggiore tanto l'oggetto è vicino

Fate un test, collimate, tenendo chiuso un occhio il pollice a qualcosa di molto lontano. E poi chiudete/aprite alternativamente gli occhi. Potete notare come il pollice si sposti significativamente nelle due viste (è relativamente vicino a voi), mentre il punto lontano no...

- Given a stereo pair of rectified images, disparity is the different of the horizontal coordinates of correspondent “points”
 - The closer the object, the larger the disparity
 - The farther the object, the smaller the disparity



Quello che si nota è che gli stessi oggetti si trovano in posizioni leggermente differenti tra destra e sinistra

E la differenza di posizione DISPARITÀ è tanto maggiore tanto l'oggetto è vicino

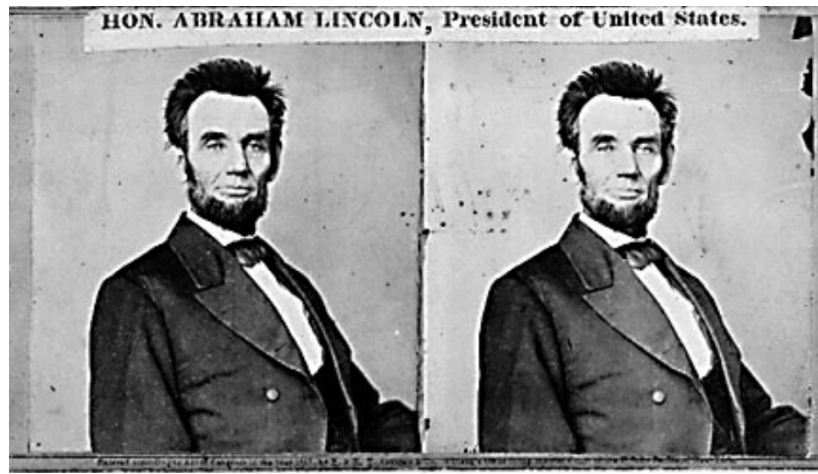
Fate un test, collimate, tenendo chiuso un occhio il pollice a qualcosa di molto lontano. E poi chiudete/aprite alternativamente gli occhi. Potete notare come il pollice si sposti significativamente nelle due viste (è relativamente vicino a voi), mentre il punto lontano no...

Disparity is useful?



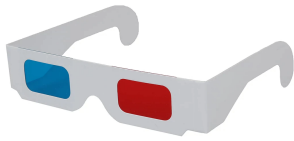
Da due immagini stereo ricostruiamo benissimo il 3D

Disparity is useful?



Da due immagini stereo ricostruiamo benissimo il 3D

Disparity is useful?

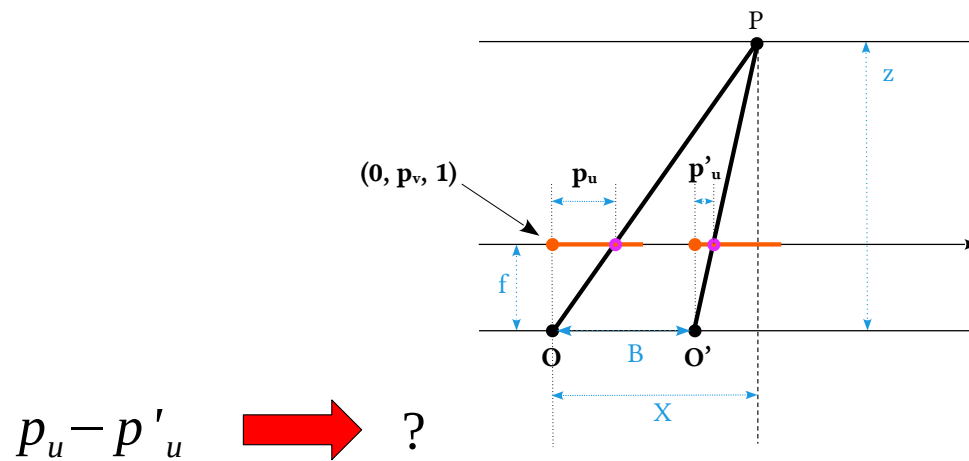


Da due immagini stereo ricostruiamo benissimo il 3D

Disparity



Disparity



$B \rightarrow$ baseline

$f \rightarrow$ focale

$U \rightarrow$ linea epipolare

L'immagine parte in realtà a sinistra rispetto la O

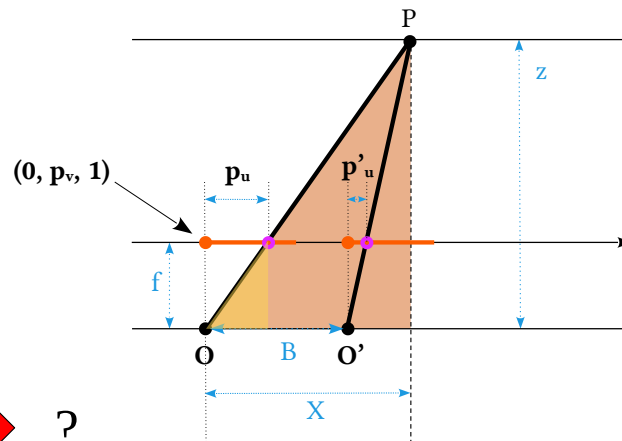
Disparity

The **yellow** right triangle features f and p_u as sides

The **brown** one features Z and X

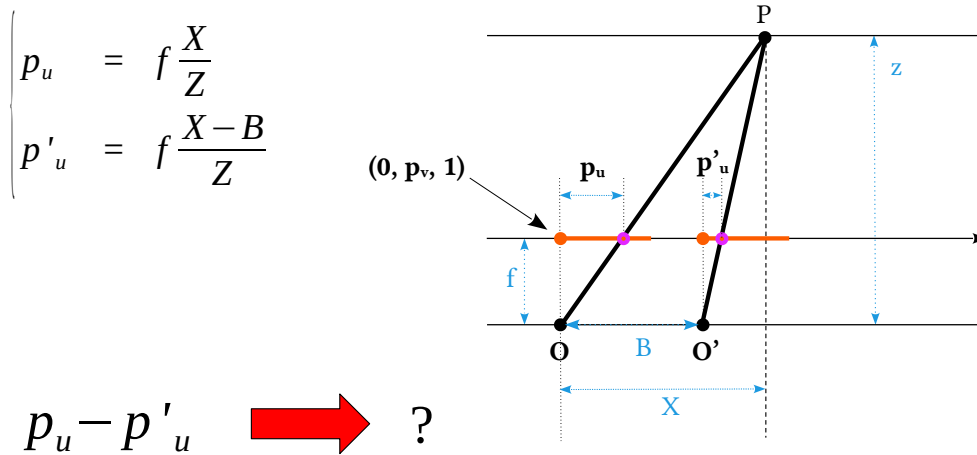
$$p_u = f \frac{X}{Z}$$

$$p_u - p'_u \rightarrow ?$$



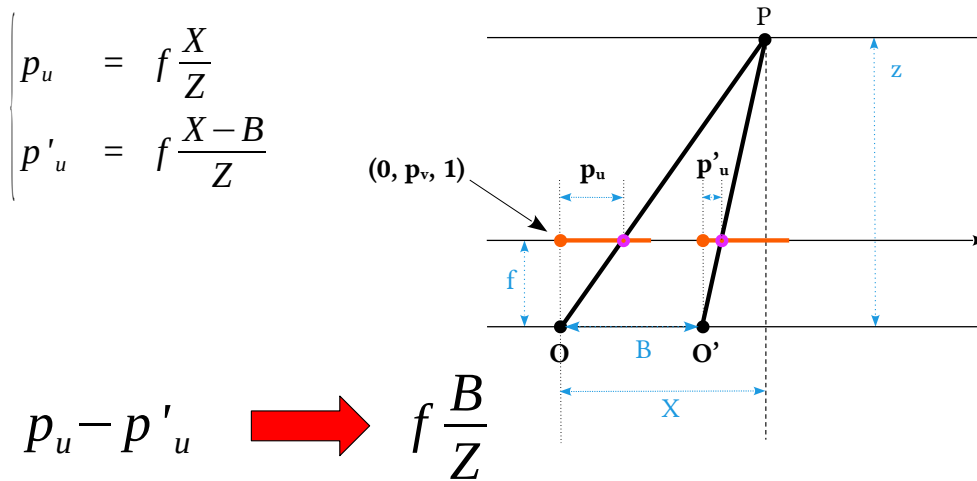
Usando triangoli simili ricaviamo la relazione che lega proiezioni e punto mondo.
Che poi è quella che abbiamo visto nel caso pin-hole

Disparity



Per l'immagine destra vale la stessa relazione, la X' la posso vedere però come $X - (O' - O) = X - B$

Disparity



Mettendo insieme il tutto ricavo che

La disparità $d = p_u - p'_u = f \frac{B}{Z}$

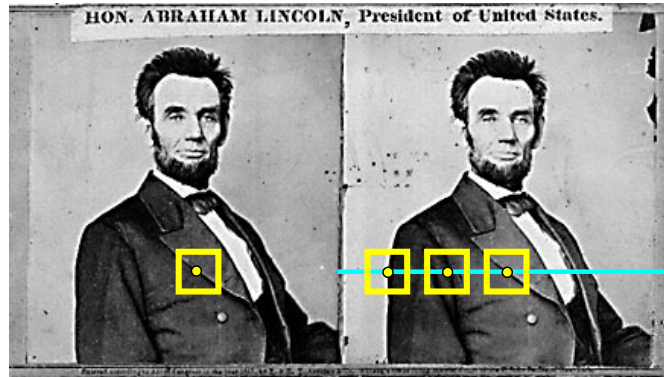
MA attenzione che l'unica incognita qui sopra è la Z!

- The $p_u - p'_u$ “difference” is the disparity d
- Disparity is inversely proportional to Z
- Yes it can give us information about depth!

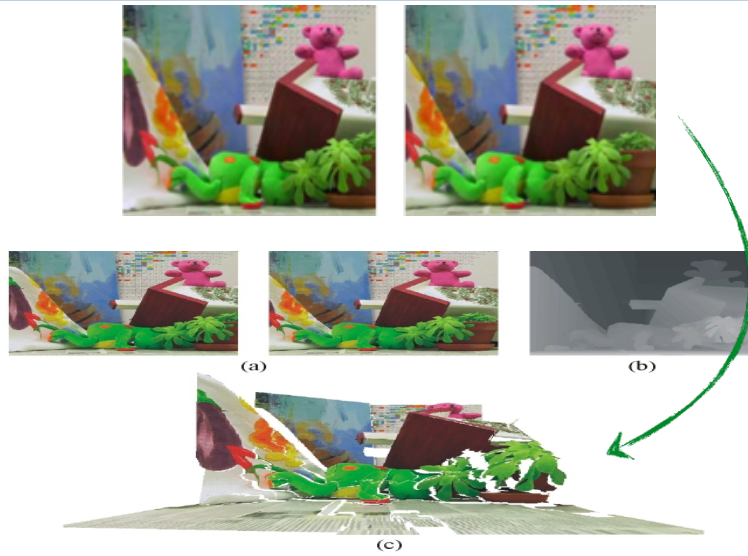
$$p_u - p'_u \quad \longrightarrow \quad d = f \frac{B}{Z}$$

U è la linea epipolare in questione da cui P_u

- Simple steps
 - Rectify images (strictly speaking not mandatory, but foul to not do that)
 - For each pixel
 - (Find epipolar line)
 - Find best match
 - Estimate Z



Depth estimation



Find best match

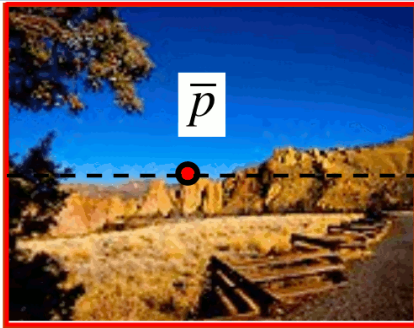


image 1

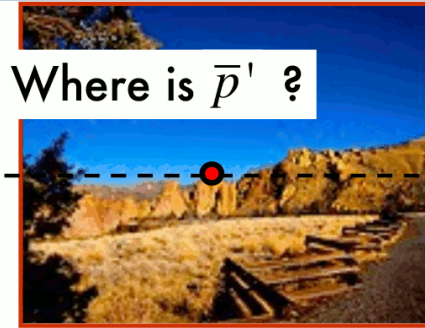
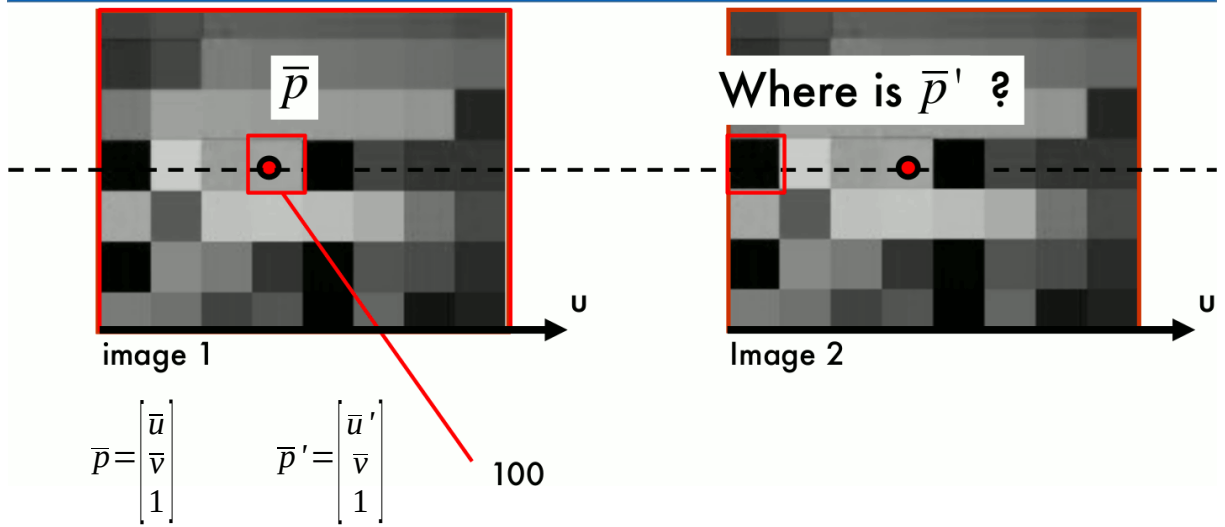


Image 2

$$\bar{p} = \begin{bmatrix} \bar{u} \\ \bar{v} \\ 1 \end{bmatrix} \quad \bar{p}' = \begin{bmatrix} \bar{u}' \\ \bar{v} \\ 1 \end{bmatrix}$$

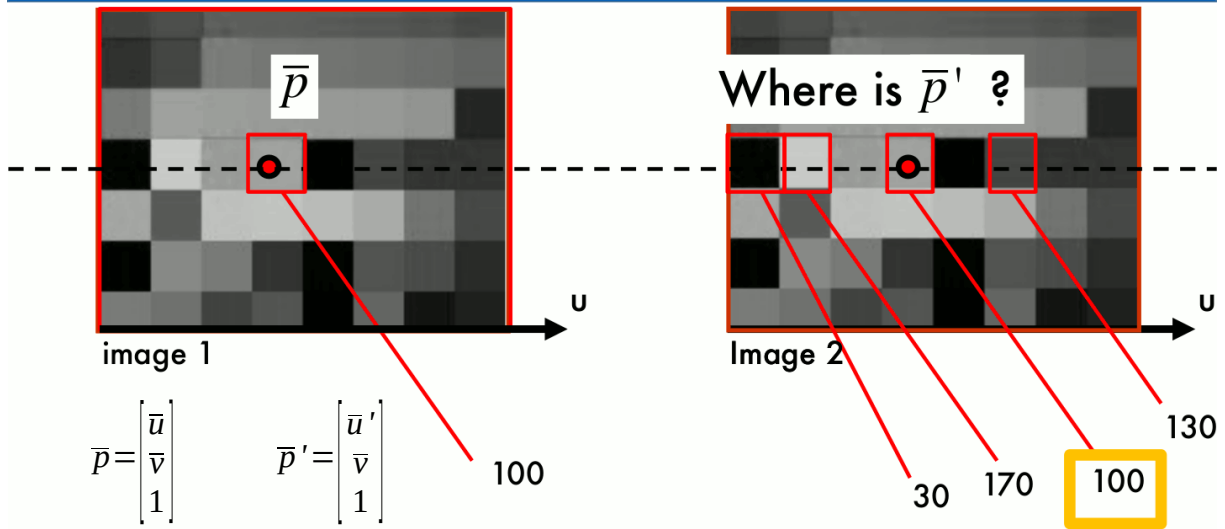
Dato un p devo cercare p' (o viceversa). Se le immagini sono rettificare sicuramente hanno la stessa coordinata verticale v .

Find best match



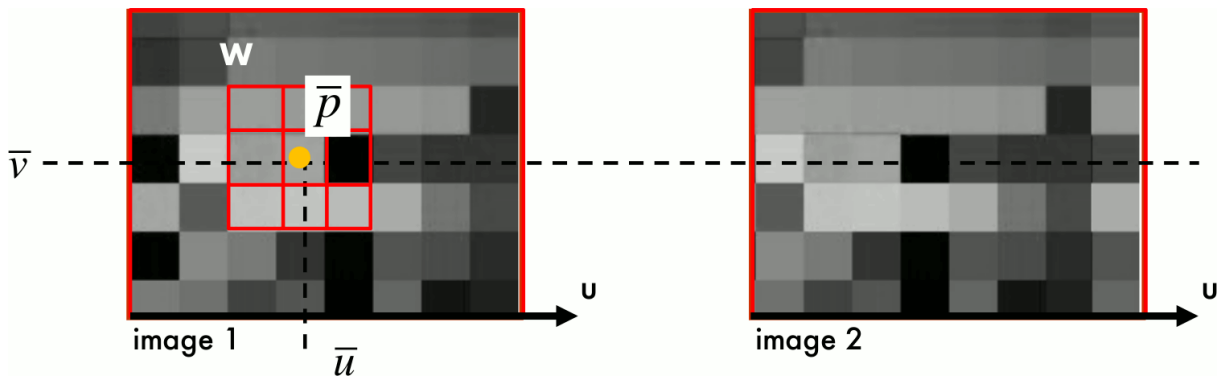
Usare il solo valore del pixel p per cercare p' non è particolarmente affidabile

Find best match



Usare il solo valore del pixel p per cercare p' non è particolarmente affidabile. A parte che il rumore e la relativa trasformazione di rettificazione introducono errori, possono comunque esserci tanti pixel che hanno un valore simile o uguale a quanto cerco.

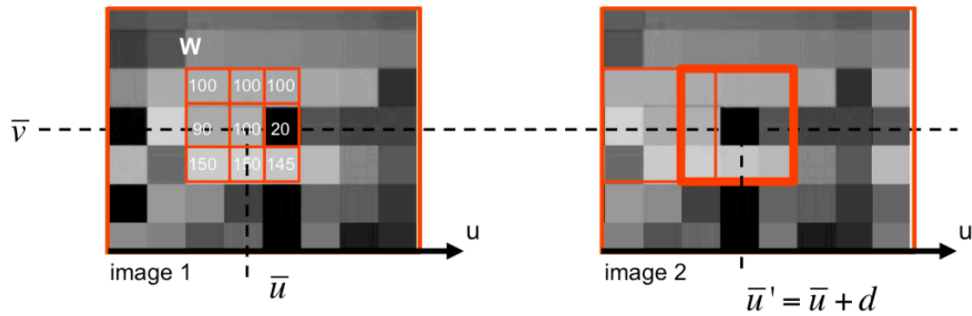
Find best match



- We use a W window around $\bar{p}=(\bar{u},\bar{v})$
- Search for the same block in the other image

Quindi in generale si usa una finestra (più o meno ampia) attorno al pixel.
Negli esempi successivi si usa una finestra 3x3 per semplicità, non assumiate che sia sempre così però.

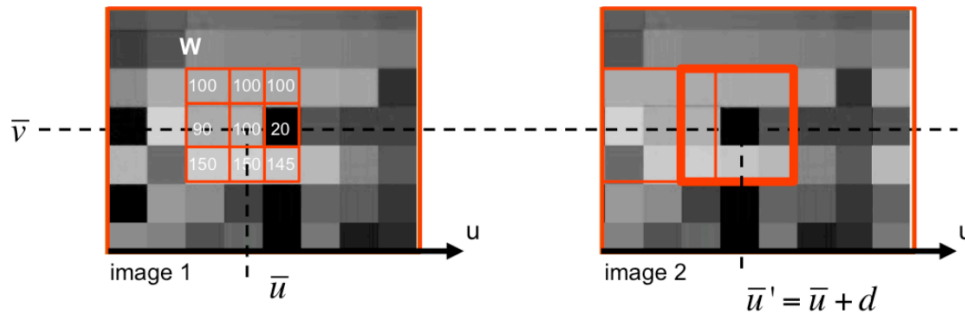
Find best match



- Run along the same line $\bar{v}=\bar{v}'$ and compute $\mathbf{W}'$ for each position
- Match $\mathbf{W}$ vs $\mathbf{W}' \rightarrow$ how we can do that?

Fissiamo una finestra $\mathbf{W}$ immagine 1 e compariamola con tutte le possibili $\mathbf{W}'$ nell'altra immagine sulla stessa linea quindi stesso $\bar{v}$ ma $\bar{u}'=\bar{u}+d$
Ok, ma come facciamo il paragone?

Cross Correlation



Example: W is a 3x3 window in red

w is a 9x1 vector

$w = [100, 100, 100, 90, 100, 20, 150, 150, 145]^T$

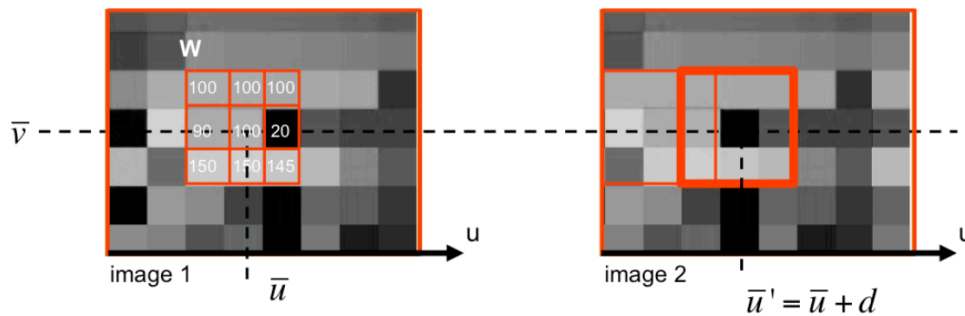
- Compute dot product $w^T \cdot w'(u')$ for acceptable u' 's and compute max
 - Max correlation (considering w and w' as vectors from W and W')

Data una finestra con n valori (nel nostro caso sono 9) posso vedere quei valori come le componenti di un vettore in uno spazio a n dimensioni.

E comparare quindi le finestre usando il prodotto scalare tra vettori, il prodotto scalare è massimo quando trovo un vettore con gli stessi valori (o proporzionali).

Rammentiamo che di fatto è il prodotto dei moduli per il coseno dell'angolo.

Cross Correlation



Example: $\mathbf{W}$ is a 3x3 window in red

$\mathbf{w}$ is a 9x1 vector

$\mathbf{w} = [100, 100, 100, 90, 100, 20, 150, 150, 145]^T$

- Highly affected by intensity variations
- Mismatches!

Non funziona benissimo, neanche questo (certo, meglio del confronto pixel a pixel)

In practice, however, for a number of reasons that we will discuss later (occlusions, foreshortening, illumination conditions, exposure etc.), the maximum value may not necessarily occur at $\bar{u} + d$ but at different u coordinate, which makes the correlation method fail. In this case we say that correlation method fails to find the right match (corresponding point) and produces a mismatch.

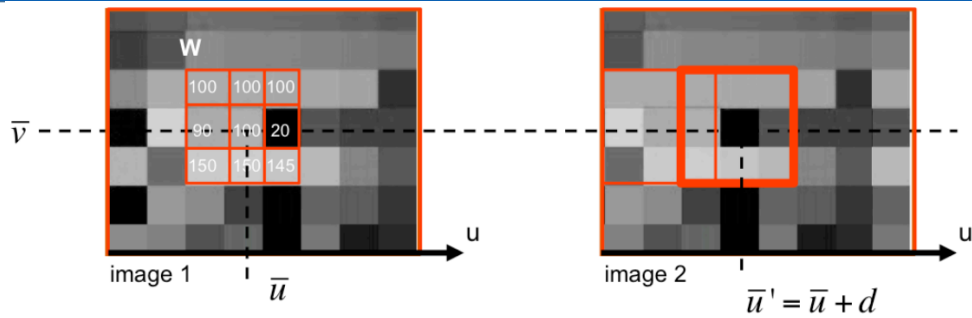


Il guadagno automatico o l'autoexposure ci possono fregare. La scena è la stessa, ma è bastato inquadrare con la telecamera dx zone più chiare che sono cambiati i parametri di acquisizione. Quindi il match diventa difficile se non ne tengo conto.

Assumendo, ma non è esattissimo, che ci sia un semplice +qualcosa in quella più chiara, se normalizzo forse ne traggio vantaggio.

Here we want address one common case that takes place when the mean and the variance of intensity values in corresponding windows change. This is often due to differences in the camera exposure conditions in image 1 and 2, for instance. A possible way for partially undoing these effects is to normalize the image windows as we compute the correlation – this is called the normalized cross correlation.

NCC: Normalized Cross Correlation



Find u' that maximize

$$\frac{(w - \bar{w})^t (w'(u') - \bar{w}'(u'))}{\|w - \bar{w}\| \|w'(u') - \bar{w}'(u')\|}$$

$\bar{w}$ = mean value within W
located at $\bar{u}$ in image 1

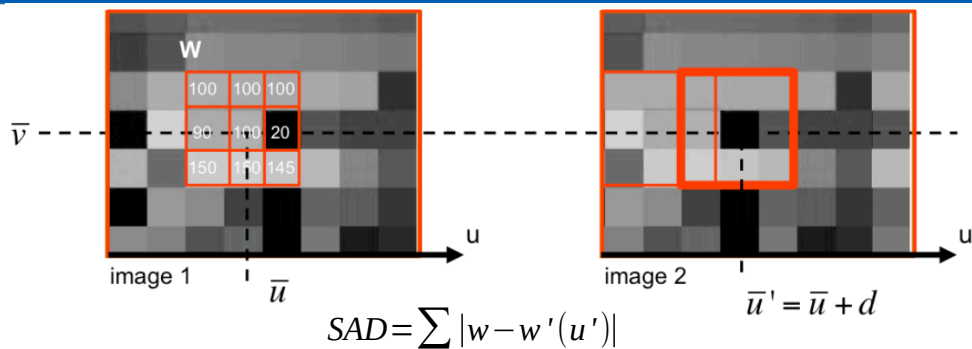
$\bar{w}'(u')$ = mean value within W
located at $\bar{u}'$ in image 2

È una crosscorrelazione normalizzata. Tengo conto di differenti luminosità.

Di fatto è massima per vettori uguali ma sostanzialmente sfasati.

Funziona? Sì, ma è onerosa da calcolarsi.

SAD: Sum of Absolute Distances



- Sum of absolute differences between W and W' (or w and $w'(u')$)
- Winner Takes All $\rightarrow$ only get the best match
- Easy to use other windows formats: squares, rectangles, adaptive...

Ha il pregio di essere molto più “rapida” da calcolarsi

WTA è spesso usato ma altri approcci sono possibili... e anzi ne parleremo

- Complexity: $O(WHDW_{\text{win}}H_{\text{win}})$
- Memory
 - No buffer required
- Highly dependent on window size
- Sparse maps can be efficiently computed
 - Compute stereo on features only

Ragioniamo sulla complessità

Assumendo:

- W e H dimensioni immagini
- D massima disparità
- Wwin e Hwin dimensioni finestra di matching

Io devo comunque centrare la mia finestrella di ricerca su tutti I punti dell'immagine sx e quindi $\sim W \times H$ punti

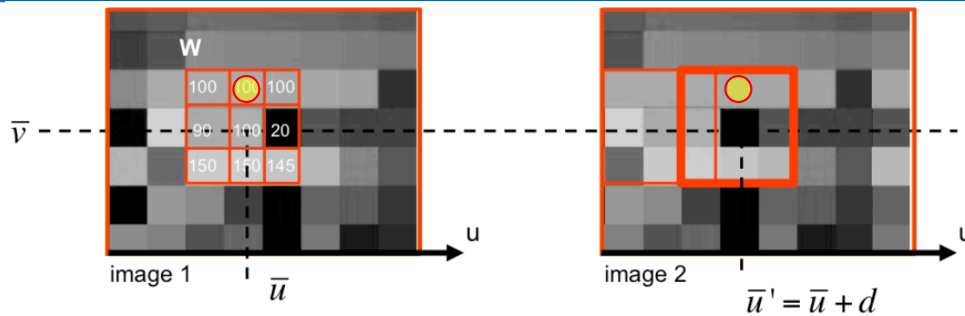
Poi devo provare fino a D possibili finestre equivalenti nell'altra immagine $\rightarrow$ moltiplichiamo per D

Per ciascun match possibile devo calcolare SAD (o simile) e quindi mi devo passare tutti I punti delle due finestre (una a sx e man mano quelle a dx che testo) ovvero moltiplichiamo ancora per Wwin e Hwin

È un numero enorme... vantaggio $\rightarrow$ algoritmo semplice e non mi devo appoggiare a specifici buffer aggiuntivi di memoria

Le mappe sparse si usano quando non elaboro tutta l'immagine ma solo dove c'è un contenuto significativo o comunque di interesse

SAD: Optimization Trick 1



- Given a specific coordinate set $\{\bar{v}, \bar{u}, \bar{u}'\}$, I have to match the elements of W like the pixels under the yellow circle
- It is the first time, I have to “compare” them?
- No, I had to “compare” them also for $\{\bar{v} - 1, \bar{u}, \bar{u}'\}$
 - If I store the result somewhere, no need to recompute $\rightarrow$ Semi incremental algo

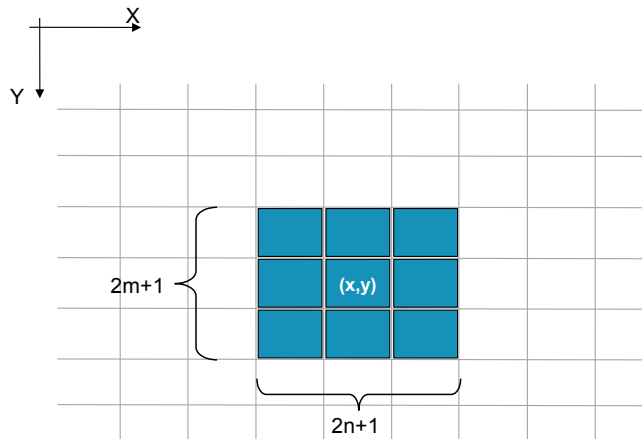
Nel caso della SAD la comparazione è di fatto il calcolo di differenze assolute.

Sì può notare che alcuni di questi calcoli li ripeto un sacco di volte. Ad esempio dei pixel a destra e sinistra indicati con il pallino giallo ho già calcolato la differenza assoluta.

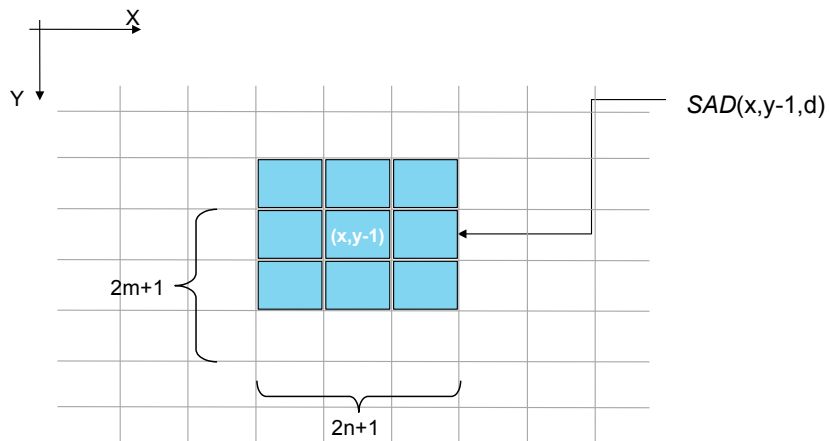
Ad esempio quando mi calcolavo la SAD per I pixel della riga sopra a quella in esame (ovvero riga di coordinate $v-1$)

A parità di u e u' in questo caso quei due pixel sarebbero stati I pixel centrali delle due finestre W e W'

Quindi posso usare operazioni già svolte quando calcolavo la disparità per la riga precedente.

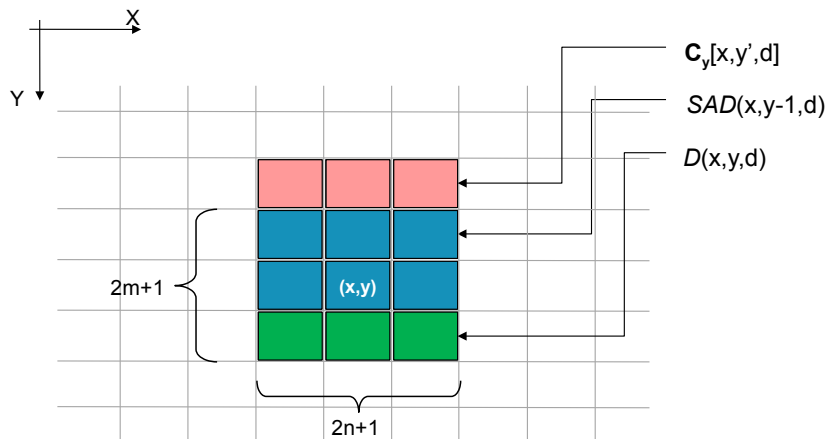


Siamo sulla riga y e vogliamo calcolare il match per uno dei punti di quella riga. Dovremmo calcolare la differenza assoluta (AD) di tutti i punti della mia patch attorno a quel punto



LA riga prima $(y-1)$ però siamo passati sempre su quella colonna quindi le differenze dei pixel tra immagine destra e sinistra di alcuni di quei pixel le abbiamo calcolate

SAD: Semi-incremental Algo



Se sovrapponiamo le due patch possiamo considerare tre zone:

- Zona sovrapposta (blu, differenze già calcolate al giro precedente)
- Zona del match con $y-1$ ma non sovrapposta a quella che devo calcolare adesso (anche qui le differenze le ho già calcolate)
- Zona nuova (verde, da calcolare)

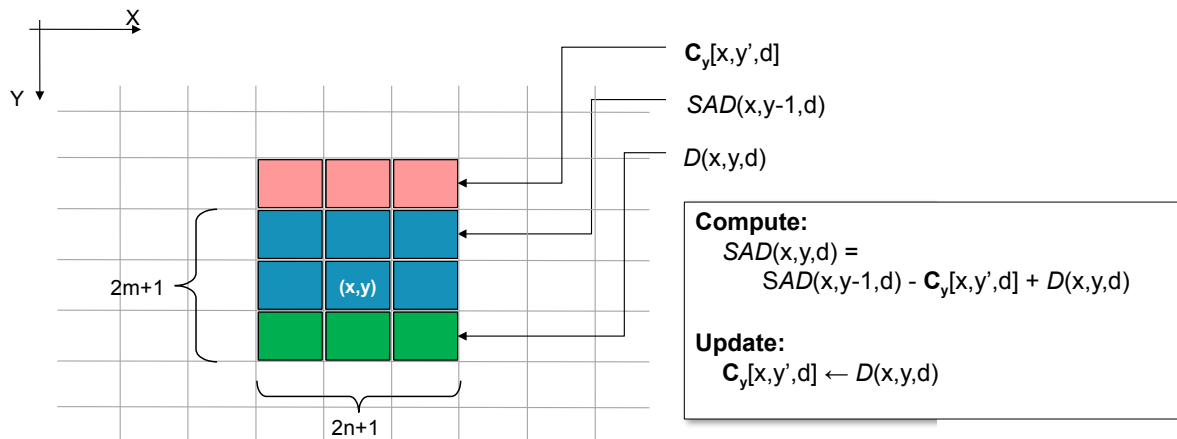
Quindi in pratica ho già calcolato buona parte di quelle differenze.

Il trucco è memorizzarsele da qualche parte per poi usarle successivamente

In pratica mi memorizzo la SAD per $y-1$, la riga in alto della stessa SAD (C_y) e mi calcolo solo le differenze nella zona verde (una sola riga in pratica)

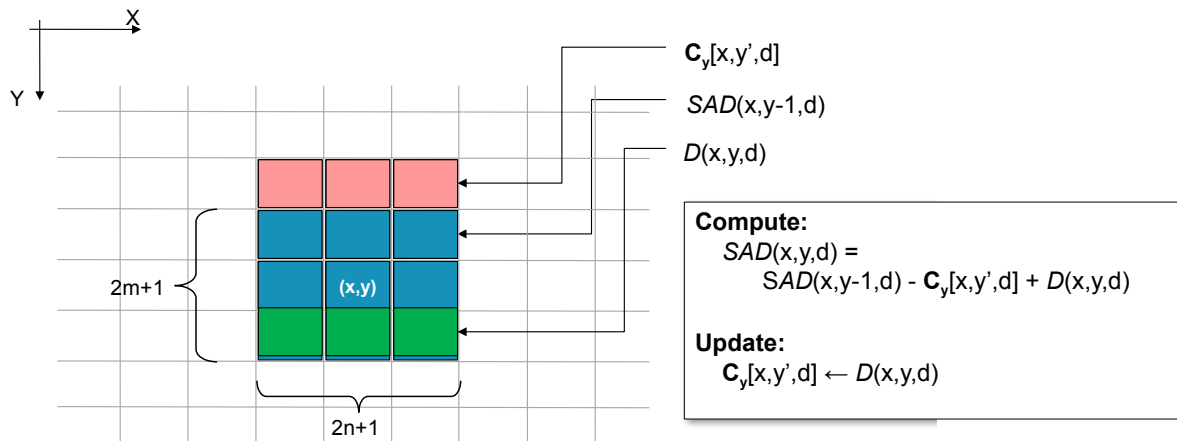
Nota: per la C_y è indicata come riga y' in quanto dipende dalle dimensioni della patch, per il disegno in esame sarebbe $y-2$ in quanto è stata disegnata una patch 3×3

SAD: Semi-incremental Algo



La SAD alla riga y la posso vedere quindi come la somma della SAD alla riga $y-1$ (non la devo calcolare, già memorizzata), meno la C_y (anche questa me la ero memorizzata), più la SAD della sola riga verde (e questa me la devo calcolare)

SAD: Semi-incremental Algo



Slide con animazione

- Complexity: $O(WHDW_{win})$
- Memory
 - C_y : WDH_{win} size vector
 - SAD: WD vector
- Sparse maps less efficiently computed

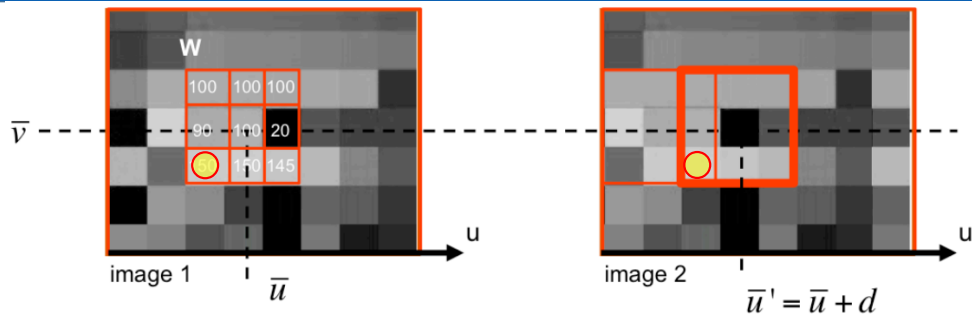
Tralasciamo il fatto che per le prime righe i calcoli li devo comunque fare tutti.

Però a regime di fatto mi limito a calcolare una riga per volta della patch. Ecco perché il mio $O()$ si riduce.

Ogni volta però memorizzo tale riga come C_y e le varie SAD calcolate. Dal punto di vista della memoria non è poco.

Se poi in una immagine devo calcolare la disparità solo in alcune aree l'efficienza si riduce visto che per ciascuna area, per le prime righe devo fare tutto

SAD: Optimization Trick 2



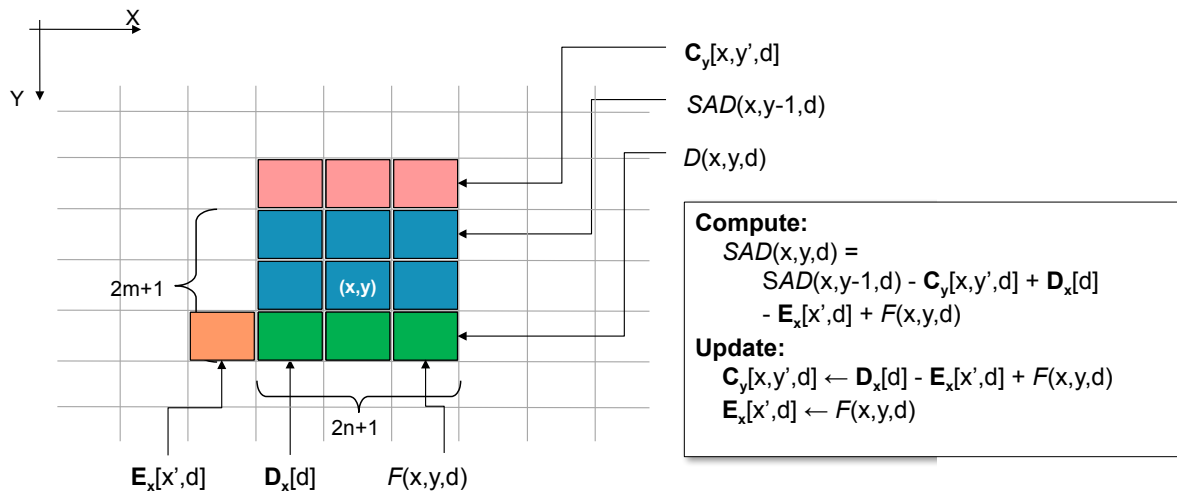
- Again, given a specific coordinate set $\{\bar{v}, \bar{u}, \bar{u}'\}$, I have to match the elements of W like the pixels under the yellow circle
- Again, it is the first time, I have to “compare” them?
- No, I had to “compare” them also for $\{\bar{v}, \bar{u}, \bar{u}'-1\}$
 - If I store the result somewhere, no need to recompute $\rightarrow$ Semi incremental algo

Discorso simile al precedente. Prendo un punto dell'ultima riga. Il semi incremental algo appena visto mi dice che per quella riga io mi devo calcolare le differenze.

Effettivamente alla riga $v-1$ non avevo toccato quell'area.

Però anche qui è una differenza che avevo già calcolato quando avevo calcolato la patch nella colonna precedente...

SAD: Full-incremental Algo



Non vi faccio vedere tutti i passaggi in quanto il discorso è del tutto simile al precedente.

Quando calcolo la SAD per la colonna prima (-1) mi memorizzo parte dell'ultima riga. O meglio, l'ultima riga e il pixel tutto a sx (E_x).

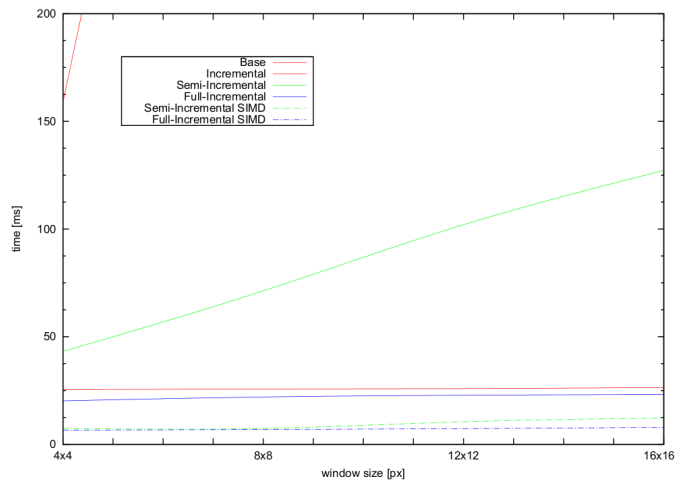
Applico quindi formula simile alla precedente.

- Complexity: $O(WHD)$
- Memory
 - C_y : WDH_{win} size vector
 - E_x : W_{win} size vector
 - SAD: WD vector
- Sparse maps can not be efficiently computed
- Border problems

All'atto pratico, a regime (ovvero salvo le prime righe e le prime colonne), calcolo la differenza di un solo pixel.

Devo memorizzare molta più roba però

SAD approaches comparison



- 512x384 disparity map on a Intel Core i7@2.66 GHz

SAD: S



Credits: Brown et al.

MATCH METRIC	DEFINITION
Normalized Cross-Correlation (NCC)	$\frac{\sum_{u,v} (I_1(u,v) - \bar{I}_1) \cdot (I_2(u+d,v) - \bar{I}_2)}{\sqrt{\sum_{u,v} (I_1(u,v) - \bar{I}_1)^2 \cdot \sum_{u,v} (I_2(u+d,v) - \bar{I}_2)^2}}$
Sum of Squared Differences (SSD)	$\sum_{u,v} (I_1(u,v) - I_2(u+d,v))^2$
Normalized SSD	$\sum_{u,v} \left(\frac{(I_1(u,v) - \bar{I}_1)}{\sqrt{\sum_{u,v} (I_1(u,v) - \bar{I}_1)^2}} - \frac{(I_2(u+d,v) - \bar{I}_2)}{\sqrt{\sum_{u,v} (I_2(u+d,v) - \bar{I}_2)^2}} \right)^2$
Sum of Absolute Differences (SAD)	$\sum_{u,v} I_1(u,v) - I_2(u+d,v) $
Rank	$\sum_{u,v} (I'_1(u,v) - I'_2(u+d,v))$ $I'_k(u,v) = \sum_{m,n} I_k(m,n) < I_k(u,v)$
Census	$\sum_{u,v} HAMMING(I'_1(u,v), I'_2(u+d,v))$ $I'_k(u,v) = BITSTRING_{m,n}(I_k(m,n) < I_k(u,v))$

Brown, M. Z., Burschka, D., and Hager, G. D. 2003.
 Advances in Computational Stereo. IEEE Trans. Pattern Anal.
 Mach. Intell. 25, 8 (Aug. 2003), 993-1008.

SAD: S

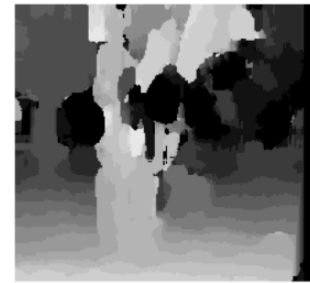
Credits: Brown et al.

REAL-TIME SYSTEM	IMAGE SIZE	FRAME RATE	RANGE BINS	METHOD	PROCESSOR	CAMERAS
INRIA 1993	256x256	3.6 fps	32	Normalized Correlation	PeRLe-1	3
CMU iWarp 1993	256x240	15 fps	16	SSAD	64 Processor iWarp Computer	3
Teleos 1995	320x240	0.5 fps	32	Sign Correlation	Pentium 166 MHz	2
JPL 1995	256x240	1.7 fps	32	SSD	Datacube & 68040	2
CMU Stereo Machine 1995	256x240	30 fps	30	SSAD	Custom HW & C40 DSP Array	6
Point Grey Triclops 1997	320x240	6 fps	32	SAD	Pentium II 450 MHz	3
SRI SVS 1997	320x240	12 fps	32	SAD	Pentium II 233 MHz	2
SRI SVM II 1997	320x240	30+ fps	32	SAD	TMS320C60x 200MHz DSP	2
Interval PARTS Engine 1997	320x240	42 fps	24	Census Matching	Custom FPGA	2
CSIRO 1997	256x256	30 fps	32	Census Matching	Custom FPGA	2
SAZAN 1999	320x240	20 fps	25	SSAD	FPGA & Convolvers	9
Point Grey Triclops 2001	320x240	20 fps 13 fps	32	SAD	Pentium IV 1.4 GHz	2 3
SRI SVS 2001	320x240	30 fps	32	SAD	Pentium III 700 MHZ	2

Brown, M. Z., Burschka, D., and Hager, G. D. 2003.
Advances in Computational Stereo. IEEE Trans. Pattern Anal.
Mach. Intell. 25, 8 (Aug. 2003), 993-1008.



Window size = 3



Window size = 20

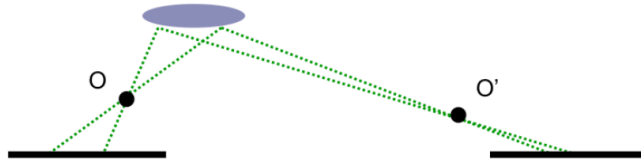
- Small windows: more details $\leftrightarrow$ more noise
- Bigger windows: less details $\leftrightarrow$ less noise
 - disparity map is more uniform

Ocio che con finestre grandi il rischio è che non abbia match!

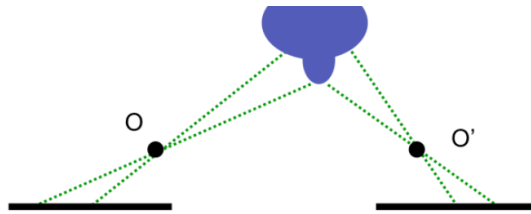
The size of the window plays a critical role in determining the effectiveness of correlation methods. A smaller window results in increasing the ability to resolve small details at the cost of adding more noise, as there is less information to reliably match corresponding points. A larger window allows points to be matched with confidence, but sacrifices detail. In the two top right images we see the disparity maps for a window of size 3 (notice that details of the tree are nicely resolved, but many mismatches occur) and for a window of size 20 (notice that the resolution of the reconstruction is poor, but most of the mismatched have been removed). We can recognize mismatches in those images because mismatched points are estimated with the wrong depth (disparity) and thus are shown with the “wrong” gray code.

Stereo is a difficult task!

- Fore shortening effects



- Occlusions

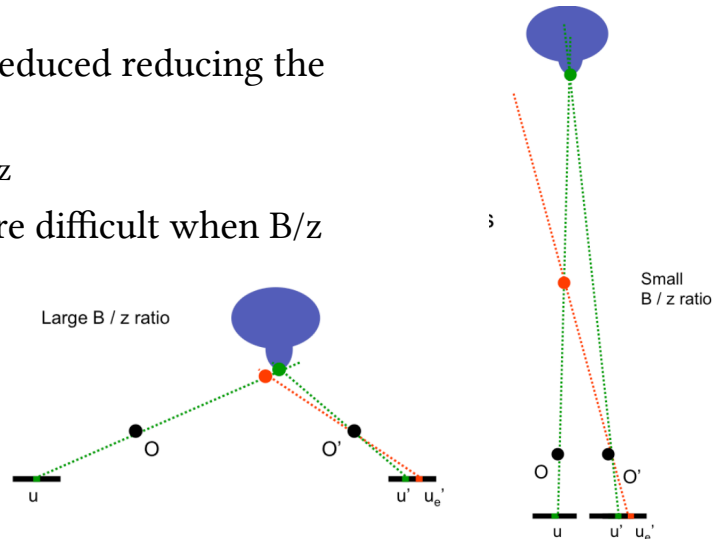


Fireshortening → angoli di vista molto differenti → area (dimensioni) diversa su immagini dx/sx

Occlusioni, sx e dx vedono cose differenti

Stereo is a difficult task!

- Those effects can be reduced reducing the baseline
 - Actually reducing B/z
- Depth estimation more difficult when B/z ratio is low!
 - Large errors



I problemi precedenti si riducono entrambi con baseline piccole. Ma ridurre B significa aumentare l'incertezza sul z calcolato

A volte sistemi trinoculari non simmetrici proprio per giocare su coppie con baseline differenti

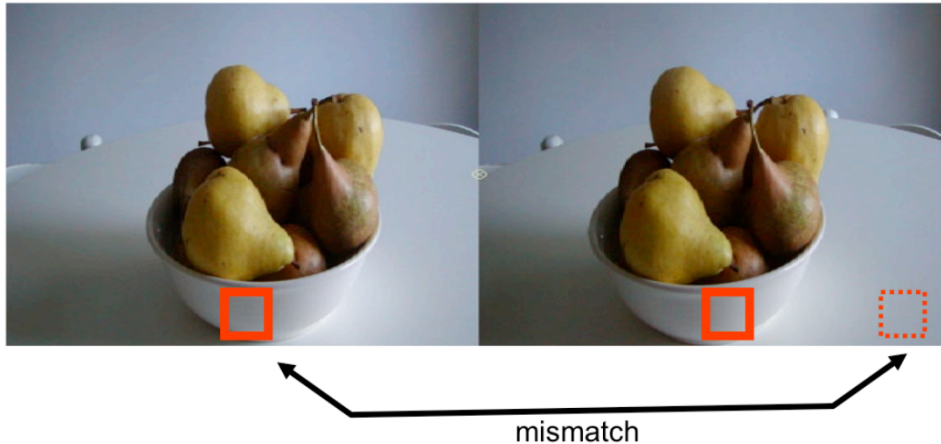
With small baselines or large depths, the matching process is less likely to be affected by foreshortening and occlusions. Thus, it is desirable to have small baseline-depth ratios. However, as such ratio decreases, we also have that the stereo depth estimation process becomes very sensitive to errors in estimating corresponding points. This is because the depth of a point in 3D is obtained by triangulating (intersecting) the two lines of sights corresponding to p and p' . Clearly, if B is small such lines of sights are almost parallel which implies that small errors in estimating the disparity yield large errors in computing the point of intersection of the two lines. As the figures show, when the baseline B is large (left), a small error in estimating the matched point in the second image (e.g., ue' instead of u') implies that the corresponding estimated 3D point (shown in red) is not too far from the actual one (shown in green). When the baseline is small, a small error in estimating the matched point in the second

Trinocular systems



Stereo is a difficult task!

- Homogeneous regions



E poi ci siamo detti, prendiamo dove la SAD o la funzione che uso matcha meglio. Sì ma avrò comunque match sbagliati...

Ad esempio regioni omogenee mi possono indurre in errori anche con lievi differenze di illuminazione

An other source of problems comes from homogeneous image regions – that is, regions in the image that share very similar intensity value distributions. Homogeneous image regions will produce a flat correlation response in which it is very hard to find correspondences. The maxima of the response will be essentially random, so spurious matches will be found, resulting in a noisy disparity map. For instance, the red bounding box (window) on left (that should match the red bounding box on the right), can be mismatched with the window in dashed red on the right because all these windows share very similar intensity distributions.

Stereo is a difficult task!

- Repetitive patterns



Anche i pattern ripetitivi mi possono indurre in errore. La patch, ad esempio, centrata sugli incroci di quelle sbarre avrà ottime corrispondenze ad ogni incrocio. E visto che il background un po' differisce, non è detto vinca quello giusto.

Repetitive patterns also pose a challenge to correlation methods. Here, the similarity response will contain multiple peaks corresponding to each repetition of the pattern, with no guarantee the right repetition will be chosen for any given correspondence.

Stereo is a difficult task!

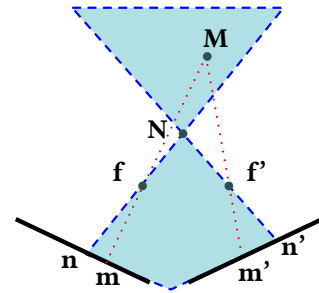
- Different view points → fore shortening
- Occlusions
- Baseline trade-off
- Homogeneous regions
- Repetitive patterns

- More issues
 - Transparent objects
 - Reflections

Ecc ecc.

Stereo is a difficult task!

- How can we cope with those issues?
- Use non local constraints
 - Uniqueness
 - Only one match
 - Continuity
 - Disparity should not change too quickly (except borders)
 - Ordering
 - Matches should have the same order in both views



The closer object N implies a forbidden zone. Any point in such region (like M) has projections that violate the ordering constraint

Come risolvo o comunque affronto quei problemi?

Ad esempio quando vado a fare il match non mi accontento di dove sia migliore ma guardo anche gli altri in classifica. Se ci sono troppi match simili al migliore mi arrendo (uniqueness)

Poi mi aspetto che la z ovvero la disparità vari con relativa continuità. Se ho salti di z improvvisi, li elimino (salvo che non siano bordi, che quindi mi devo calcolare)

L'idea di base dell'ordering è che oggetti più vicini debbano nascondere gli oggetti più lontani, che proiettivamente appaiono in ordine diverso nelle due immagini

- $(p_u, p_v, d) \rightarrow 3D$

$$d = p_u - p'_u \Rightarrow z_c = \frac{B \cdot f}{d}$$

- Camera reference system

$$\begin{matrix} p_u & \xrightarrow{\quad} & x_c \\ p_v & \xrightarrow{\quad} & y_c \\ d & & z_c \end{matrix}$$

$$\begin{cases} x' = p_u - u_0 = f \frac{x_c}{z_c} \\ y' = p_v - v_0 = f \frac{y_c}{z_c} \end{cases}$$



$$\begin{cases} x_c = \frac{(p_u - u_0) \cdot B}{d} \\ y_c = \frac{(p_v - v_0) \cdot B}{d} \\ z_c = \frac{B \cdot f}{d} \end{cases}$$

Dato il punto e la disparità d calcolata in corrispondenza mi calcolo facilmente le coordinate di quel punto nel sistema di riferimento della camera (qui in versione semplificata)

$$\begin{bmatrix} R & T \\ 0 & 1 \end{bmatrix}^T \cdot \begin{bmatrix} x_c \\ y_c \\ z_c \\ 1 \end{bmatrix} = \begin{bmatrix} Y_w \\ Y_w \\ Z_w \end{bmatrix}$$

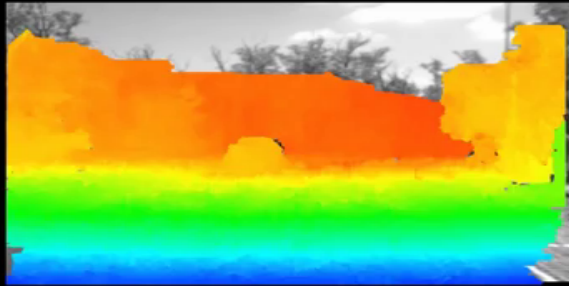
- RT can help us to compute world coordinates

E poi posso usare gli estrinseci per riportarlo in altro sistema di riferimento

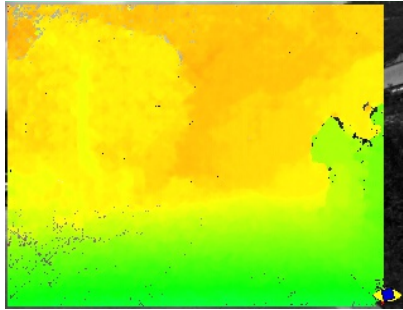
Recap about depth estimation

- We need to calibrate both cameras
 - Compute F or E
- Rectify images $\rightarrow$ parallel image planes
- Search for correspondences
- Transform disparity in world coordinates

Depth from disparity



- Semi Global Matching allows to obtain **dense** disparity maps
 - Cost function based on disparity directions



Esempio di come un approccio che usa Semi Global Matching (SGM) mi permette di riempire i “buchi” e ottenere una disparità densa.

In condizioni normali è un problema NP completo ma tramite assunzioni si semplifica

